

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для управления ассортиментом и
заказами магазина цифровой электроники»

Выполнил: Кузнецов Данила Ростиславович

Группа: 241-327

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Кузнецову Даниле Ростиславовичу,

обучающемуся группы 241-327,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для управления ассортиментом и заказами магазина цифровой электроники»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих интернет-магазинов электроники и систем управления заказами	10	22.03.2026	
Задача 1.2. Анализ программных средств разработки веб-приложений (Flask, Django, FastAPI)	7	26.03.2026	
Задача 1.3. Формулировка цели, задач и требований к проекту	8	30.03.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории (покупатели, менеджеры, администраторы)	5	02.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.2. Описание функциональности приложения	7	06.04.2026	
Задача 2.3. Проектирование модели данных (товары, категории, пользователи, заказы, корзина)	8	10.04.2026	ER-диаграмма
Задача 2.4. Разработка макетов страниц и пользовательских сценариев	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка пользовательского интерфейса	8	21.04.2026	
Задача 3.2. Реализация аутентификации пользователей и разграничения прав доступа	7	27.04.2026	RBAC
Задача 3.3. Реализация CRUD-интерфейса для товаров, категорий и заказов	7	03.05.2026	REST API
Задача 3.4. Реализация корзины, оформления заказа и истории покупок	12	09.05.2026	
Задача 3.5. Реализация поиска, фильтрации и сортировки товаров	6	15.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Развертывание приложения и базы данных	4	26.05.2026	Docker
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «30» марта 2026 г.

Дата сдачи выполненной работы «13» июня 2026 г.

Задание принял к исполнению

«30» марта 2026 г.


(подпись)

Кузнецов Данила
Ростиславович

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих интернет-магазинов электроники и систем управления заказами	6
1.2 Анализ программных средств разработки веб-приложений ..	8
1.3 Формулировка цели, задач и требований к проекту	11
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	14
2.1 Анализ целевой аудитории	14
2.2 Описание функциональности приложения	14
2.3 Проектирование модели данных	15
2.4 Разработка макетов страниц и пользовательских сценариев .	17
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	19
3.1 Разработка базовой структуры приложения и вёрстка интер- фейса.....	19
3.2 Проектирование моделей данных (пользователи, товары и заказы)	19
3.3 Реализация корзины товаров и управления количеством	20
3.4 Оформление заказа и аутентификация	21
3.5 Реализация поиска, фильтрации и сортировки товаров	22
3.6 Разграничение прав доступа и панель менеджера	23
ЗАКЛЮЧЕНИЕ	24
СПИСОК ЛИТЕРАТУРЫ	25

ВВЕДЕНИЕ

Актуальность темы. Рынок цифровой электроники характеризуется большим количеством товарных позиций, быстрым обновлением моделей и высокой зависимостью покупательского выбора от технических характеристик. Для магазина электроники важно не только представить каталог, но и обеспечить своевременное обновление цен, остатков, категорий и статусов заказов. Ручное ведение таких данных увеличивает вероятность ошибок, затрудняет обработку заявок и снижает качество обслуживания покупателей. Поэтому разработка веб-приложения для управления ассортиментом и заказами магазина цифровой электроники является актуальной практической задачей.

Степень разработанности проблемы. Современные интернет-магазины и маркетплейсы предоставляют пользователям развитые средства поиска, фильтрации, оформления заказа и отслеживания доставки. Однако готовые коммерческие платформы часто ограничивают владельца магазина правилами конкретной площадки, комиссионной моделью или закрытой логикой управления. Для небольшого магазина цифровой электроники актуальным остается создание самостоятельного веб-приложения, которое учитывает особенности ассортимента, роли покупателей, менеджеров и администраторов, а также требования к обработке заказов.

Объект исследования — процессы управления онлайн-продажами цифровой электроники, включая ведение ассортимента, взаимодействие покупателей с каталогом и обработку заказов.

Предмет исследования — программные средства и архитектурные решения для разработки веб-приложения, автоматизирующего управление товарами, категориями, пользователями, корзиной и заказами магазина цифровой электроники.

Целью данной работы является проектирование и реализация веб-приложения для управления ассортиментом и заказами магазина цифровой электроники, обеспечивающего покупателям удобную работу с каталогом и заказами, а администраторам — инструменты управления товарами, категориями и заявками.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих интернет-магазинов электроники и систем управления заказами

Рынок цифровой электроники относится к числу наиболее динамичных сегментов электронной коммерции. Покупатели ожидают от интернет-магазина не только удобного каталога товаров, но и подробных технических характеристик, сравнения моделей, актуальной информации о наличии, понятного оформления заказа, выбора способа доставки и прозрачного отслеживания статуса покупки. Для администрации магазина важны другие задачи: управление ассортиментом, контроль остатков, обработка заказов, обновление цен, работа с категориями, пользователями и историей продаж.

В качестве аналогов были рассмотрены крупные интернет-магазины и платформы, работающие с цифровой электроникой: DNS, Ситилинк и Ozon. Эти решения позволяют выявить типовые функциональные возможности, которые целесообразно учитывать при проектировании собственного веб-приложения.

DNS — российская торговая сеть и интернет-магазин цифровой и бытовой техники. Каталог DNS содержит смартфоны, ноутбуки, комплектующие, периферию, бытовую электронику и аксессуары. Сильной стороной платформы является развитая структура категорий, фильтры по техническим характеристикам и карточки товаров с описанием, ценой, наличием в магазинах, отзывами и вариантами доставки. Для покупателя доступны корзина, оформление заказа, личный кабинет и история покупок. Однако внутренняя система управления заказами ориентирована на процессы самой сети и не может быть использована как самостоятельный инструмент для небольшого магазина электроники.

Ситилинк представляет собой крупный интернет-магазин электроники, компьютерной техники и товаров для дома. Платформа поддерживает поиск по каталогу, подбор товаров по характеристикам, сравнение моделей, оформление доставки или самовывоза, а также работу с корпоративными клиентами. Система хорошо демонстрирует важность точного описания характеристик товара: для цифровой электроники покупатель часто принимает решение на основании параметров процессора, объема памяти, диагонали экрана, емкости аккумулятора и совместимости аксессуаров. Ограничением с точки зрения

проектируемого решения является закрытость бизнес-логики: магазин не предоставляет сторонним продавцам гибкий модуль управления собственным ассортиментом и заказами.

Ozon является универсальным маркетплейсом, где цифровая электроника представлена как одна из крупных товарных категорий. Платформа предоставляет покупателю развитые инструменты поиска, фильтрации, сортировки, просмотра отзывов и отслеживания доставки. Для продавцов существует личный кабинет с управлением товарами, ценами, остатками и заказами. При этом работа через маркетплейс предполагает зависимость от правил площадки, комиссий, ограничений интерфейса и общей логики обработки заказов. Для небольшого магазина, которому требуется самостоятельный сайт с собственной базой клиентов и управлением продажами, такой подход не всегда является оптимальным.

Сравнительная характеристика рассмотренных решений приведена в таблице 1.1.

Таблица 1.1 – Сравнительный анализ интернет-магазинов цифровой электроники

Название	Ассортимент	Управление заказами	Личный кабинет	Гибкость для малого бизнеса
DNS	Смартфоны, ноутбуки, комплектующие, периферия, бытовая техника	Оформление покупки, выбор доставки или самовывоза, отслеживание статуса	История заказов, избранное, данные покупателя	Система закрыта и предназначена для собственной торговой сети
Ситилинк	Электроника, компьютерная техника, комплектующие, товары для офиса и дома	Обработка заказов, самовывоз, доставка, работа с юридическими лицами	Заказы, бонусы, профиль, корпоративные функции	Готового решения для стороннего магазина не предоставляет
Ozon	Универсальный маркетплейс, включая цифровую электронику	Централизованная обработка заказов через инфраструктуру маркетплейса	История покупок, возвраты, отслеживание доставки	Возможна продажа через площадку, но с зависимостью от правил и комиссий маркетплейса

Проведенный обзор показывает, что существующие крупные платформы обладают развитым пользовательским функционалом, однако их системы управления ассортиментом и заказами либо закрыты, либо жестко связаны с инфраструктурой маркетплейса. Поэтому разработка отдельного веб-приложения для магазина цифровой электроники является актуальной: такое приложение должно объединять публичный каталог товаров, корзину, оформление заказов, личный кабинет покупателя и административный интерфейс для управления товарами, категориями и заказами.

1.2 Анализ программных средств разработки веб-приложений

Для реализации веб-приложения необходимо выбрать технологии как для серверной (backend), так и для клиентской (frontend) части.

Для серверной части веб-приложения интернет-магазина цифровой электроники были рассмотрены три популярных Python-фреймворка: Flask, Django и FastAPI. Выбор Python обусловлен простотой разработки, развитой экосистемой библиотек, удобной работой с базами данных и широким применением в учебных и коммерческих веб-проектах.

Flask — микрофреймворк, предоставляющий базовые инструменты для обработки HTTP-запросов, маршрутизации и формирования ответов. Его преимуществом является простота и гибкость: разработчик самостоятельно выбирает ORM, систему авторизации, валидацию данных и структуру проекта. Такой подход удобен для небольших приложений и прототипов. Недостатком Flask является необходимость вручную подключать и согласовывать многие компоненты, которые в интернет-магазине требуются практически всегда: регистрацию пользователей, разграничение прав, административный интерфейс, работу с формами и защиту операций.

Django — полнофункциональный веб-фреймворк, ориентированный на быструю разработку надежных серверных приложений. Он включает ORM для работы с базой данных, систему миграций, встроенную административную панель, механизмы аутентификации, работу с формами и шаблонами. Для проекта интернет-магазина эти возможности особенно важны, поскольку требуется хранить товары, категории, пользователей, корзины и заказы, а также предоставить администратору удобный интерфейс управления данными. Django уменьшает объем вспомогательного кода и позволяет сосредоточиться

на предметной логике магазина.

FastAPI — современный фреймворк для создания API, основанный на типизации Python и автоматической генерации документации OpenAPI. Он хорошо подходит для высокопроизводительных REST API и асинхронной обработки запросов. FastAPI удобен, если проект строится как отдельный backend для SPA-приложения или мобильного клиента. При этом для классического интернет-магазина с административной панелью и большим количеством стандартных CRUD-операций часть инфраструктуры потребуется подключать и настраивать отдельно.

Сравнение фреймворков по ключевым критериям приведено в таблице 1.2.

Таблица 1.2 – Сравнение Flask, Django и FastAPI

Критерий	Flask	Django (выбран)	FastAPI
Архитектура	Минимальное ядро, высокая гибкость	Полнофункциональный фреймворк с готовой структурой	API-ориентированный подход с поддержкой асинхронности
Работа с базой данных	Требуется подключение SQLAlchemy или другого ORM	Встроенная ORM и миграции	ORM выбирается отдельно, часто используется SQLAlchemy
Администрирование	Готовая панель отсутствует	Встроенная административная панель	Административный интерфейс требуется реализовывать отдельно
Аутентификация и права доступа	Подключаются расширениями	Встроенные пользователи, группы и права	Реализуются через дополнительные библиотеки и собственную логику
Подходящая область применения	Небольшие сервисы и прототипы	Интернет-магазины, CRM, панели управления, CRUD-системы	Высоконагруженные API и сервисы для SPA/мобильных клиентов

На основании сравнения для реализации проекта выбран Django. Его встроенные средства позволяют быстрее разработать основные модули интернет-магазина: каталог товаров, категории, пользователей, корзину, оформление

заказа и административное управление. Кроме того, Django хорошо подходит для проектов, где важны целостность данных, разграничение доступа и поддерживаемая структура приложения.

В качестве базы данных целесообразно использовать реляционную СУБД SQLite на этапе разработки и PostgreSQL при последующем развертывании. Реляционная модель удобна для описания связей между товарами, категориями, пользователями, заказами и позициями заказа. Для обмена данными между клиентской и серверной частью может применяться формат JSON, особенно при реализации REST API для каталога, корзины и заказов.

Для реализации клиентской части (frontend) были рассмотрены следующие технологии: классическая связка HTML/CSS/JavaScript с использованием шаблонизаторов, а также современные JavaScript-фреймворки (React, Vue.js).

HTML/CSS/JavaScript (с шаблонами Django) — традиционный подход, при котором сервер генерирует готовые HTML-страницы. Этот вариант отличается простотой интеграции с Django, быстрой первоначальной загрузкой и хорошей SEO-оптимизацией. Дополнительно можно использовать CSS-фреймворки (например, Bootstrap или Tailwind CSS) для ускорения вёрстки адаптивного интерфейса.

React — популярная библиотека для создания пользовательских интерфейсов на основе компонентов. React отлично подходит для разработки сложных интерактивных Single Page Application (SPA), обеспечивая высокую скорость отклика без перезагрузки страниц. Однако его использование требует настройки отдельного фронтенд-проекта, сборщика модулей (Webpack/Vite) и организации обмена данными с сервером через REST API.

Vue.js — прогрессивный JavaScript-фреймворк, который можно использовать как для создания полноценных SPA, так и для постепенного внедрения интерактивности на отдельные страницы. Он обладает более низким порогом вхождения по сравнению с React и предлагает удобную систему реактивности, но также требует дополнительных трудозатрат на настройку API-взаимодействия.

Сравнение технологий клиентской части приведено в таблице 1.3.

Таблица 1.3 – Сравнение технологий клиентской части

Критерий	Шаблоны Django + JS	React	Vue.js
Архитектура	Многостраничное приложение (MPA)	Одностраничное приложение (SPA)	SPA или внедрение в MPA
Сложность интеграции с Django	Минимальная (встроено во фреймворк)	Высокая (требуется REST API и отдельная сборка)	Средняя (возможна постепенная интеграция)
Интерактивность	Базовая (обновления через перезагрузку страниц или AJAX)	Высокая (динамическое обновление компонентов)	Высокая
Подходящая область применения	Информационные сайты, простые магазины, админ-панели	Сложные интерактивные веб-приложения и сервисы	Гибкие проекты, где важен баланс между простотой и реактивностью

На основании сравнения, для реализации пользовательского интерфейса интернет-магазина выбран подход с использованием **HTML/CSS/JavaScript** и шаблонизатора **Django**, дополненный CSS-фреймворком Bootstrap для адаптивной вёрстки. Этот выбор позволит сократить время на разработку базового функционала магазина (каталог, корзина, оформление заказа) и избежать избыточной сложности, связанной с поддержкой двух отдельных кодовых баз (frontend и backend), что оптимально подходит для масштаба курсового проекта.

1.3 Формулировка цели, задач и требований к проекту

На основе анализа предметной области и рассмотренных программных средств была сформулирована цель курсовой работы.

Цель работы — разработать веб-приложение для управления ассортиментом и заказами магазина цифровой электроники, обеспечивающее покупателям удобную работу с каталогом и заказами, а администраторам — инструменты ведения товаров, категорий и обработки заявок.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ существующих интернет-магазинов электроники и

определить основные функции, необходимые для проектируемого приложения.

2. Выбрать программные средства разработки серверной части веб-приложения с учетом требований к каталогу, заказам, пользователям и административному управлению.
3. Спроектировать структуру данных для хранения товаров, категорий, пользователей, корзины, заказов и позиций заказа.
4. Реализовать пользовательский интерфейс для просмотра каталога, поиска, фильтрации, добавления товаров в корзину и оформления заказа.
5. Реализовать аутентификацию пользователей и разграничение ролей покупателей, менеджеров и администраторов.
6. Создать CRUD-интерфейс для управления товарами, категориями и заказами.
7. Провести тестирование основных сценариев работы приложения и проверить корректность обработки пользовательских действий.

К проектируемому веб-приложению предъявляются следующие **функциональные требования**:

- отображение каталога цифровой электроники с разделением товаров по категориям;
- хранение карточек товаров с названием, описанием, ценой, изображением, характеристиками и количеством на складе;
- поиск, фильтрация и сортировка товаров;
- регистрация и авторизация пользователей;
- добавление товаров в корзину и изменение количества позиций;
- оформление заказа с сохранением состава покупки и контактных данных;
- просмотр пользователем истории своих заказов;

- управление товарами, категориями и заказами через административный интерфейс;
- разграничение прав доступа для покупателей, менеджеров и администраторов.

Также необходимо учитывать **нефункциональные требования**: интерфейс должен быть понятным для покупателей без специальной подготовки, данные заказов должны сохраняться в структурированном виде, действия администратора должны быть защищены авторизацией, а архитектура приложения должна допускать дальнейшее расширение, например добавление промокодов, отзывов, интеграции с платежной системой или службами доставки.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Целевая аудитория разрабатываемого веб-приложения включает три основные группы пользователей: покупателей, менеджеров магазина и администраторов системы.

Покупатель использует приложение для просмотра каталога цифровой электроники, поиска и фильтрации товаров, изучения характеристик, добавления товаров в корзину и оформления заказа. Для данной группы важны понятная навигация по категориям, быстрый поиск по названию и характеристикам, корректное отображение цены и наличия товара.

Менеджер отвечает за обработку заказов, изменение их статусов, проверку состава покупки и коммуникацию с клиентом. Для менеджера особенно важны список новых заказов, информация о покупателе, составе заказа, общей сумме и текущем состоянии выполнения.

Администратор управляет товарным каталогом, категориями, пользователями и правами доступа. Его рабочие сценарии связаны с добавлением новых моделей электроники, редактированием цен, описаний, изображений, складских остатков и характеристик товаров.

2.2 Описание функциональности приложения

Проектируемое приложение должно поддерживать основные сценарии интернет-магазина цифровой электроники. Покупатель просматривает каталог, выбирает категорию, применяет фильтры, открывает карточку товара, добавляет товар в корзину и оформляет заказ. После оформления заказ сохраняется в системе, а пользователь может просмотреть его состояние в истории покупок.

Менеджер просматривает поступившие заказы, уточняет их состав и изменяет статус обработки. Администратор добавляет и редактирует товары, категории и учетные записи сотрудников.

К основным функциональным модулям относятся:

- каталог товаров с категориями цифровой электроники;
- карточка товара с описанием, ценой, изображением, характеристиками и остатком;

- поиск, фильтрация и сортировка товаров;
- корзина покупателя;
- оформление и хранение заказов;
- личный кабинет покупателя с историей покупок;
- административный интерфейс управления товарами, категориями и заказами;
- разграничение прав доступа для покупателей, менеджеров и администраторов.

2.3 Проектирование модели данных

Модель данных веб-приложения строится вокруг сущностей, характерных для интернет-магазина: пользователей, категорий, товаров, корзины, заказов и позиций заказа. Между ними существуют устойчивые связи: товар относится к категории, заказ принадлежит пользователю, а каждая позиция заказа ссылается на конкретный товар.

Структура основных таблиц представлена в таблице 2.1.

Таблица 2.1 – Основные сущности модели данных

Сущность	Ключевые поля	Назначение
Пользователь	id, username, email, password, role	Хранение учетных записей покупателей, менеджеров и администраторов
Категория	id, name, slug, description	Группировка товаров по разделам каталога
Товар	id, category_id, name, description, price, stock, image, specifications	Хранение информации о цифровой электронике и ее характеристиках
Корзина	id, user_id, created_at	Временное хранение выбранных пользователем товаров

Продолжение таблицы 2.1

Сущность	Ключевые поля	Назначение
Позиция корзины	id, cart_id, product_id, quantity	Связь товара с корзиной и выбранным количеством
Заказ	id, user_id, status, total_price, delivery_address, created_at	Фиксация оформленной покупки и состояния обработки
Позиция заказа	id, order_id, product_id, quantity, price	Хранение состава заказа и цены товара на момент покупки

На рисунке 2.1 представлена ER-диаграмма базы данных, отражающая сущности системы и связи между ними.

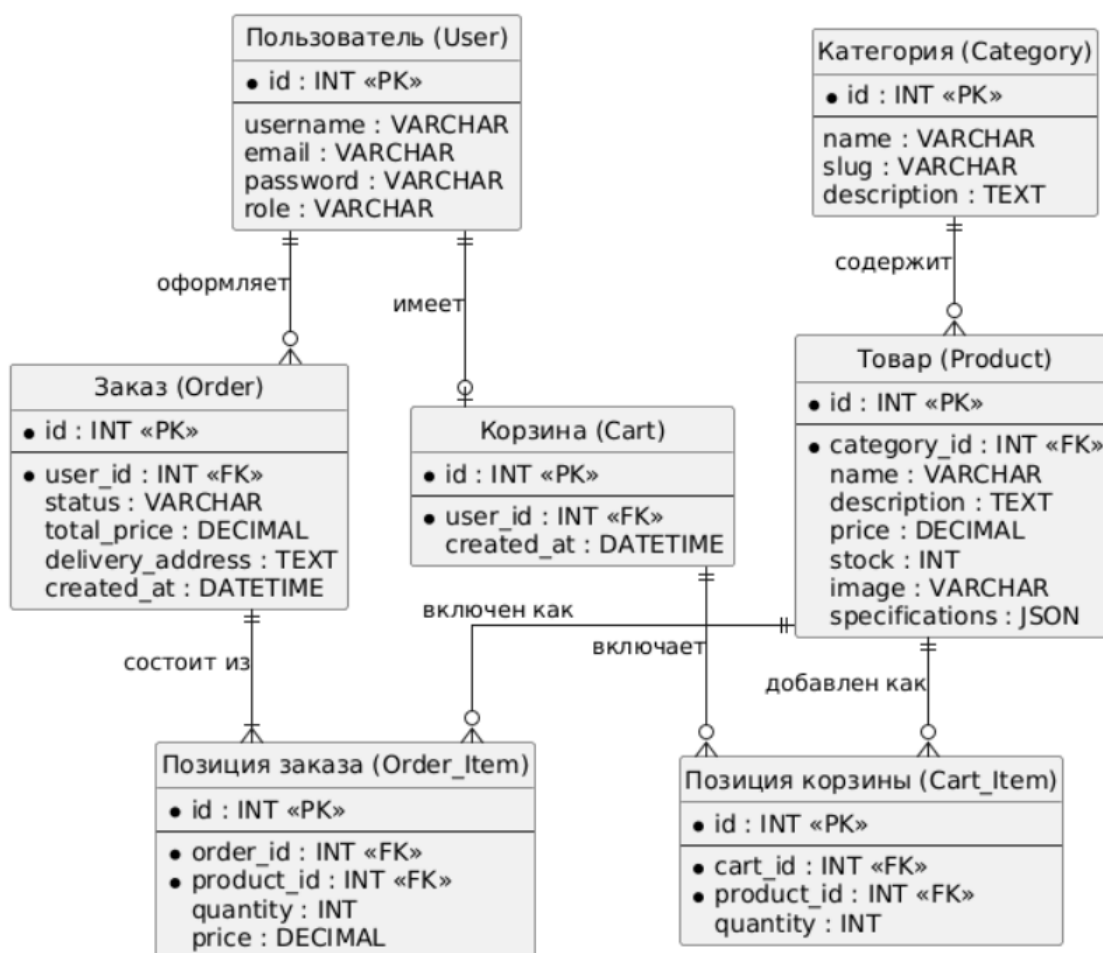


Рисунок 2.1 – ER-диаграмма базы данных интернет-магазина

2.4 Разработка макетов страниц и пользовательских сценариев

Интерфейс приложения проектируется с учетом разделения на публичную часть и административную панель. Публичная часть включает главную страницу, каталог, страницу категории, карточку товара, корзину, страницу оформления заказа и личный кабинет. Административная часть содержит списки товаров, категорий и заказов, а также формы создания и редактирования записей.

Основной пользовательский сценарий покупателя выглядит следующим образом:

1. Пользователь открывает каталог магазина цифровой электроники (см. рисунок 2.2).
2. Выбирает категорию или находит товар через поиск.
3. Изучает карточку товара и добавляет его в корзину.
4. Проверяет состав корзины (см. рисунок 2.3) и переходит к оформлению заказа.
5. Указывает контактные данные и адрес доставки.
6. Подтверждает заказ и отслеживает его статус в личном кабинете.

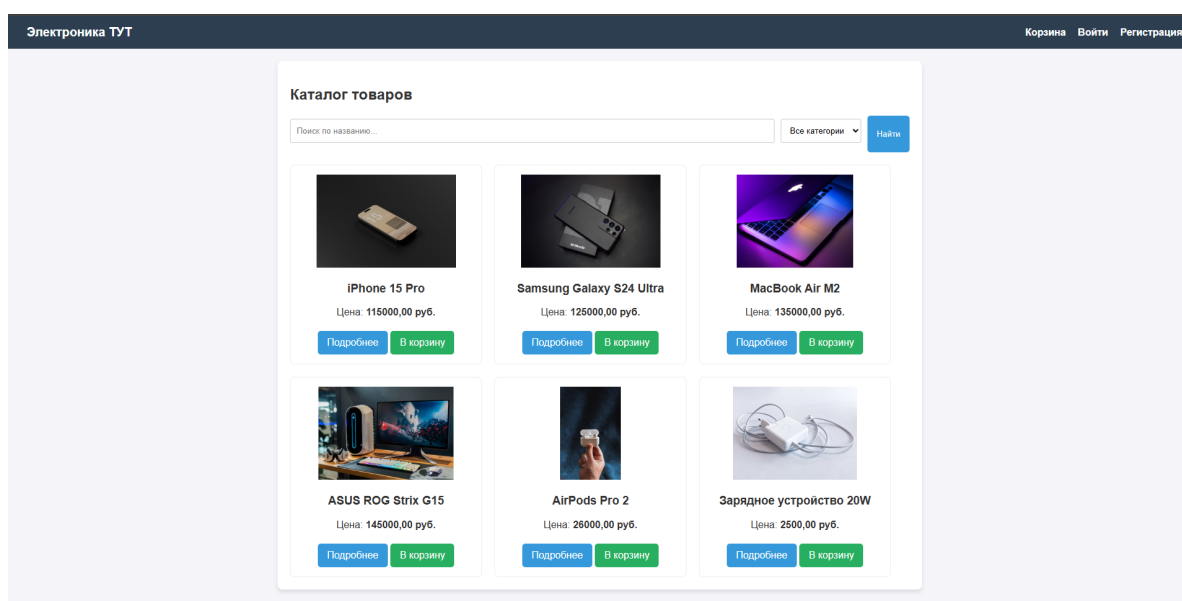


Рисунок 2.2 – Главная страница веб-приложения

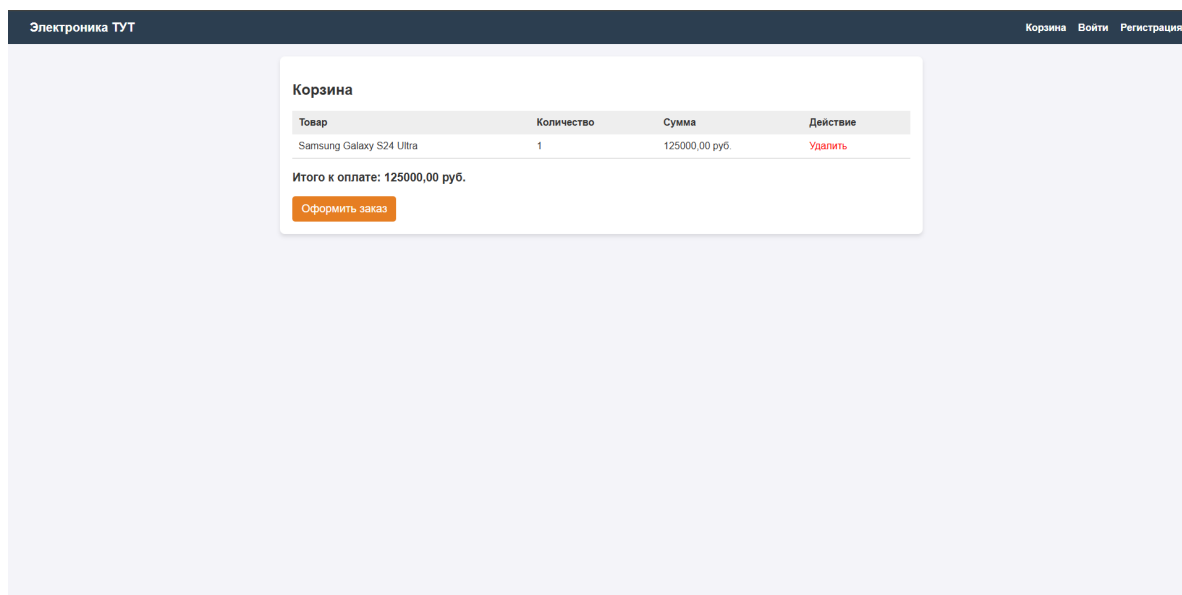


Рисунок 2.3 – Страница корзины пользователя

Такое проектирование позволяет заранее определить состав страниц, структуру данных и основные точки взаимодействия пользователя с системой.

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и вёрстка интерфейса

Разработка веб-приложения была выполнена на основе фреймворка Django. Для максимальной простоты и удобства понимания весь основной функционал магазина был вынесен в одно приложение `store`. Это позволяет избежать излишней сложности и легко отслеживать связи между товарами, корзиной и заказами.

Интерфейс спроектирован с использованием базовых HTML-шаблонов и CSS (стили встроены напрямую в базовый шаблон для простоты чтения кода). Пользователю доступны главная страница с каталогом товаров, страница с детальной информацией о товаре, корзина, оформление заказа и история покупок.

3.2 Проектирование моделей данных (пользователи, товары и заказы)

Для хранения информации в базе данных (используется надежная реляционная СУБД PostgreSQL) были созданы модели, полностью соответствующие спроектированной архитектуре. В качестве модели пользователя применяется кастомная модель `User`, расширяющая стандартный класс `AbstractUser` и включающая поле `role` для определения прав доступа (покупатель, менеджер, администратор). Основными сущностями для каталога являются `Category` (Категория) и `Product` (Товар).

Листинг 1 – Модели товаров и категорий

```
1 class Category(models.Model):
2     name = models.CharField(max_length=255, verbose_name="
    Название")
3     slug = models.SlugField(max_length=255, unique=True,
    null=True, verbose_name="Slug")
4     description = models.TextField(blank=True, verbose_name=
    "Описание")
5
6 class Product(models.Model):
7     category = models.ForeignKey(Category, on_delete=models.
    CASCADE, verbose_name="Категория")
8     name = models.CharField(max_length=255, verbose_name="
```

```

        Название")
9     description = models.TextField(verbose_name="Описание")
10    price = models.DecimalField(max_digits=10,
        decimal_places=2, verbose_name="Цена")
11    stock = models.PositiveIntegerField(default=0,
        verbose_name="Остаток на складе")
12    image = models.ImageField(upload_to='products/', blank=
        True, null=True, verbose_name="Фото")
13    specifications = models.TextField(blank=True, null=True,
        verbose_name="Характеристики")

```

Связь между ними организована по принципу "один ко многим"(одна категория может содержать много товаров). Для хранения фотографий применяется поле ImageField.

3.3 Реализация корзины товаров и управления количеством

В отличие от упрощенных реализаций, корзина в данном приложении полноценно хранится в базе данных. Для этого спроектированы две взаимосвязанные модели: Cart (Корзина), которая привязывается к пользователю, и CartItem (Позиция корзины), хранящая информацию о добавленном товаре и его количестве. Пользователь имеет возможность не только добавлять товары в корзину, но и гибко изменять их количество с помощью кнопок интерфейса. При достижении нулевого количества товар автоматически удаляется из корзины.

Листинг 2 – Динамическое изменение количества товара в корзине

```

1  @login_required(login_url='/login/')
2  def update_cart_item(request, product_id, action):
3      cart = get_object_or_404(Cart, user=request.user)
4      cart_item = get_object_or_404(CartItem, cart=cart,
        product_id=product_id)
5
6      if action == 'increase':
7          cart_item.quantity += 1
8          cart_item.save()
9      elif action == 'decrease':
10         if cart_item.quantity > 1:
11             cart_item.quantity -= 1
12             cart_item.save()
13         else:
14             cart_item.delete()

```

```
15
16     return redirect('cart_detail')
```

3.4 Оформление заказа и аутентификация

Для оформления заказа пользователь должен быть авторизован. Мы использовали кастомную модель User и создали собственную форму регистрации CustomUserCreationForm, что позволяет корректно обрабатывать новые поля, такие как роль пользователя. Авторизованный покупатель заполняет адрес доставки (delivery_address), после чего содержимое его корзины из таблиц Cart и CartItem переносится в базу данных заказов в виде Order (Заказ) и OrderItem (Позиции заказа). Корзина при этом очищается, а цены товаров жестко фиксируются на момент совершения покупки.

Листинг 3 – Оформление заказа

```
1 @login_required(login_url='/login/')
2 def checkout(request):
3     cart, _ = Cart.objects.get_or_create(user=request.user)
4     cart_items = cart.items.all()
5     if not cart_items:
6         return redirect('product_list')
7
8     if request.method == 'POST':
9         delivery_address = request.POST.get('
delivery_address')
10        order = Order.objects.create(
11            user=request.user,
12            delivery_address=delivery_address,
13            total_price=0
14        )
15        total = sum(item.product.price * item.quantity for
item in cart_items)
16
17        for item in cart_items:
18            OrderItem.objects.create(
19                order=order, product=item.product,
20                quantity=item.quantity, price=item.product.
price
21            )
22
23        order.total_price = total
24        order.save()
```

```

25         cart.items.all().delete() # Очищаем корзину после заказа
26         return redirect('order_history')
27
28     return render(request, 'store/checkout.html')

```

3.5 Реализация поиска, фильтрации и сортировки товаров

Для удобства пользователей на главной странице реализована комплексная система подбора товаров. Пользователь может осуществлять поиск по названию (icontains), фильтровать каталог по выбранной категории, а также сортировать полученный список. Доступны сортировки: от дешевых к дорогим, от дорогих к дешевым, и по алфавиту. Все параметры передаются через GET-запросы, что позволяет пользователям делиться ссылками на отфильтрованные списки.

Листинг 4 – Поиск, фильтрация и сортировка каталога

```

1 def product_list(request):
2     query = request.GET.get('q', '')
3     category_id = request.GET.get('category', '')
4     sort_by = request.GET.get('sort', '')
5
6     products = Product.objects.all()
7     if query:
8         products = products.filter(name__icontains=query)
9     if category_id:
10        products = products.filter(category_id=category_id)
11
12    if sort_by == 'price_asc':
13        products = products.order_by('price')
14    elif sort_by == 'price_desc':
15        products = products.order_by('-price')
16    elif sort_by == 'name':
17        products = products.order_by('name')
18
19    categories = Category.objects.all()
20    return render(request, 'store/product_list.html', {
21        'products': products,
22        'categories': categories,
23        'query': query,
24        'selected_category': category_id,
25        'sort_by': sort_by
26    })

```

3.6 Разграничение прав доступа и панель менеджера

В системе реализован механизм Role-Based Access Control (RBAC). При помощи поля `role` в модели пользователя система различает обычных покупателей и менеджеров магазина. Для менеджеров реализована отдельная защищенная страница управления заказами, доступ к которой обычным пользователям заблокирован. Менеджер имеет возможность просматривать все заказы, оставленные в магазине, и изменять их статус (например, переводить заказ из статуса "В обработке" в "Доставлен" или "Отменен").

Листинг 5 – Обработка заказов менеджером

```
1 @login_required(login_url='/login/')
2 def change_order_status(request, order_id):
3     if request.user.role != 'manager' and not request.user.is_superuser:
4         return HttpResponseRedirect("Доступ разрешен только менеджерам.")
5
6     if request.method == 'POST':
7         order = get_object_or_404(Order, id=order_id)
8         new_status = request.POST.get('status')
9         if new_status in dict(Order.STATUS_CHOICES):
10             order.status = new_status
11             order.save()
12
13     return redirect('manager_orders')
```

Таким образом, веб-приложение реализует весь необходимый функционал полноценного интернет-магазина, оставаясь при этом логичным, легко читаемым и соответствующим архитектуре современных веб-приложений.

Исходный код разработанного веб-приложения доступен в репозитории на платформе GitHub по ссылке: <https://github.com/DanilaKuznetsov/coursework-web-app-main>. Доступ к репозиторию предоставлен проверяющему (GitHub: akruzhalov).

Развернутая версия веб-приложения доступна в сети Интернет по адресу: <https://electronics-store-o69j.onrender.com>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была рассмотрена предметная область интернет-торговли цифровой электроникой и определены требования к веб-приложению для управления ассортиментом и заказами. Анализ существующих решений показал, что крупные интернет-магазины и маркетплейсы обладают развитым функционалом, однако не всегда подходят для самостоятельного магазина, которому требуется собственная система управления товарами, клиентами и заказами.

По результатам работы можно сформулировать следующие **выводы**:

1. Для магазина цифровой электроники ключевыми функциями являются каталог с характеристиками товаров, поиск, фильтрация, корзина, оформление заказа и история покупок.
2. Для серверной части проекта целесообразно использовать Django, так как он предоставляет встроенные средства работы с базой данных, административной панелью, пользователями и правами доступа.
3. Модель данных должна учитывать связи между пользователями, категориями, товарами, корзиной, заказами и позициями заказа, чтобы обеспечить корректную обработку покупок.
4. Разграничение ролей покупателей, менеджеров и администраторов повышает безопасность системы и упрощает организацию рабочих процессов магазина.

Перспективы развития проекта связаны с добавлением онлайн-оплаты, интеграции со службами доставки, системы отзывов, промокодов, уведомлений о статусе заказа и аналитики продаж.

При разработке пояснительной записки к курсовому проекту соблюдались требования [4], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

- [1] Django documentation. URL: <https://docs.djangoproject.com/> (дата обращения: 24.04.2026).
- [2] Flask documentation. URL: <https://flask.palletsprojects.com/> (дата обращения: 24.04.2026).
- [3] FastAPI documentation. URL: <https://fastapi.tiangolo.com/> (дата обращения: 24.04.2026).
- [4] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления. М.: Стандартинформ, 2017.